

Universidad Técnica Estatal De Quevedo (UTEQ)

Facultad de Ciencias de la Computación

Ingeniería en Software (Rediseño)

Aplicaciones Web

GA -- Práctica en clase: Proyecto ASP.NET Core MVC.

Formativa (Autoevaluación)

Presencial en el aula/laboratorio

Docente:

Dr. Gleiston Cicerón Guerrero Ulloa.

Nombre:

Zambrano Moreira Alison Ariana

Semana 7 — Del 15 al 21 de Junio del 2026

Viernes 19 de Junio 2026

URL en github: [a1l2i3s4o5n6/Pr-ctica-en-clase-Proyecto-ASP.NET-Core-MVC](https://github.com/a1l2i3s4o5n6/Pr-ctica-en-clase-Proyecto-ASP.NET-Core-MVC)

I. Introducción

El presente trabajo tiene como finalidad presentar el desarrollo de una aplicación web básica utilizando ASP.NET Core MVC. Durante la práctica se implementó un proyecto denominado MiPrimerMVC, en el cual se aplicaron los principios del patrón Modelo-Vista-Controlador (MVC) para la gestión de productos.

A lo largo del documento se describe el proceso de creación del proyecto, la implementación del modelo Producto mediante Data Annotations para la validación de datos, el desarrollo del controlador ProductoController para gestionar las operaciones de consulta y registro, y la construcción de las vistas Razor encargadas de la interacción con el usuario. Asimismo, se explica la incorporación de mecanismos de seguridad mediante AntiForgeryToken y la validación de datos tanto del lado del cliente como del servidor.

Finalmente, se presentan evidencias del funcionamiento de la aplicación, demostrando el correcto registro y validación de productos dentro del entorno ASP.NET Core MVC.

II. Creación del proyecto

Se utilizó el comando *dotnet new* con la plantilla MVC:

```
PS C:\ASP.NET CORE MVC> dotnet new mvc -n MiPrimerMVC
```

Este comando genera la estructura completa del proyecto:

Carpeta/Archivo	Propósito
Controllers/	Controladores de la aplicación
Models/	Clases de modelo (dominio)
Views/	Vistas Razor (interfaz de usuario)
Program.cs	Punto de entrada y configuración
wwwroot/	Archivos estáticos (CSS, JS, imágenes)

Estructura del proyecto generado:

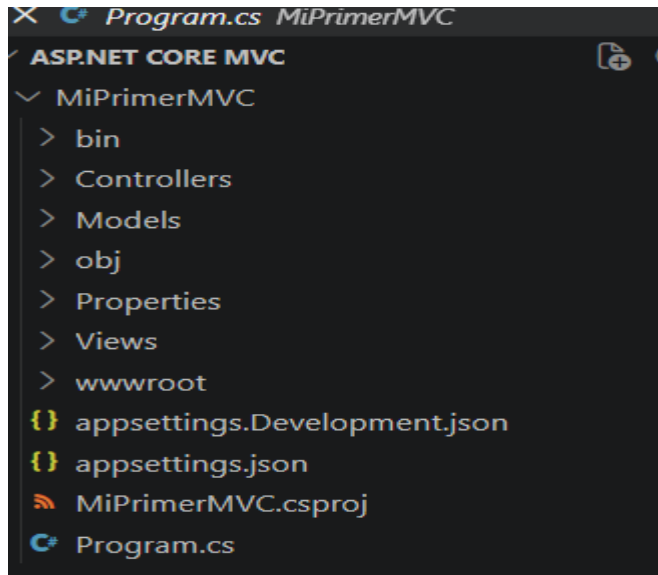


Ilustración 1 Estructura del programa

III. Modelo Producto con Data Annotations

Se creó la clase Producto en Models/Producto.cs con las siguientes propiedades y validaciones mediante Data Annotations:

Propiedad	Tipo	Data Annotations
Id	int	—
Nombre	string	[Required] [StringLength(100)]
Precio	decimal	[Required] [Range(0.01, 999999.99)] [DataType(DataType.Currency)]
Descripcion	string?	[StringLength(500)] [DataType(DataType.MultilineText)]
FechaAlta	DateTime	[Required] [DataType(DataType.Date)] [Display(Name = "Fecha de Alta")]

Models/Producto.cs

```
using System.ComponentModel.DataAnnotations;
```

```
namespace MiPrimerMVC.Models;
```

```
public class Producto
```

```

{

    public int Id { get; set; }

    [Required(ErrorMessage = "El nombre es obligatorio")]

    [StringLength(100, ErrorMessage = "El nombre no puede exceder 100 caracteres")]

    public required string Nombre { get; set; }

    [Required(ErrorMessage = "El precio es obligatorio")]

    [Range(0.01, 999999.99, ErrorMessage = "El precio debe estar entre 0.01 y 999999.99")]

    [DataType(DataType.Currency)]

    public decimal Precio { get; set; }

    [StringLength(500, ErrorMessage = "La descripción no puede exceder 500 caracteres")]

    [DataType(DataType.MultilineText)]

    public string? Descripcion { get; set; }

    [Required(ErrorMessage = "La fecha es obligatoria")]

    [DataType(DataType.Date)]

    [Display(Name = "Fecha de Alta")]

    public DateTime FechaAlta { get; set; }

}

```

Captura de la interfaz del modelo producto en visual estudio

Crear Producto

Nombre

Precio

Descripcion

Fecha de Alta

mm/dd/yyyy

Crear

Volver

Ilustración 2 Del modelo Producto en Visual Studio

IV. Controlador ProductoController

Se creó el controlador ProductoController con las acciones Index y Create, utilizando una lista estática List<Producto> para almacenar los productos en memoria.

Acción	Verbo HTTP	Descripción
Index()	GET	Muestra la lista de todos los productos.
Create()	GET	Muestra el formulario para crear un producto.
Create(Producto)	POST	Valida los datos, guarda el producto en la lista y redirige a la vista Index.

Características clave:

- ✓ Lista estática _productos compartida entre solicitudes
- ✓ Id autoincremental con _nextId
- ✓ Validación server-side con ModelState.IsValid

- ✓ [ValidateAntiForgeryToken] en el POST
- ✓ Mensaje de éxito mediante TempData
- ✓ Redirección a Index después de crear

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;

using MiPrimerMVC.Models;

namespace MiPrimerMVC.Controllers;

public class ProductController : Controller
{
    private static List<Producto> _productos = new List<Producto>();

    private static int _nextId = 1;

    [HttpGet]

    public IActionResult Index()

    {
        return View(_productos);
    }

    [HttpGet]

    public IActionResult Create()

    {
        return View();
    }

    [HttpPost]

    [ValidateAntiForgeryToken]

    public IActionResult Create(Producto producto)
```

```

{
    if (!ModelState.IsValid)
    {
        return View(producto);
    }

    producto.Id = _nextId++;

    _productos.Add(producto);

    TempData["Mensaje"] =

        $"Producto '{producto.Nombre}' creado exitosamente.";

    return RedirectToAction("Index");
}
}

```

Controlador ProductoController

El controlador ProductoController es el componente encargado de gestionar las solicitudes relacionadas con los productos y actuar como intermediario entre el modelo y las vistas.

1. Lista de productos en memoria

```
private static List<Producto> _productos = new List<Producto>();
```

Esta línea declara una lista estática donde se almacenan temporalmente los productos en memoria durante la ejecución de la aplicación.

2. Acción Index()

```
public IActionResult Index()
```

Esta acción responde a las solicitudes de la ruta /Producto y muestra una tabla con todos los productos registrados en la lista.

3. Acción Create() (GET)

```
[HttpGet]
```

```
public IActionResult Create()
```

Esta acción se ejecuta cuando el usuario selecciona la opción **Crear Nuevo Producto**. Su función es mostrar el formulario vacío para ingresar los datos del nuevo producto.

4. Acción Create(Producto producto) (POST)

[HttpPost]

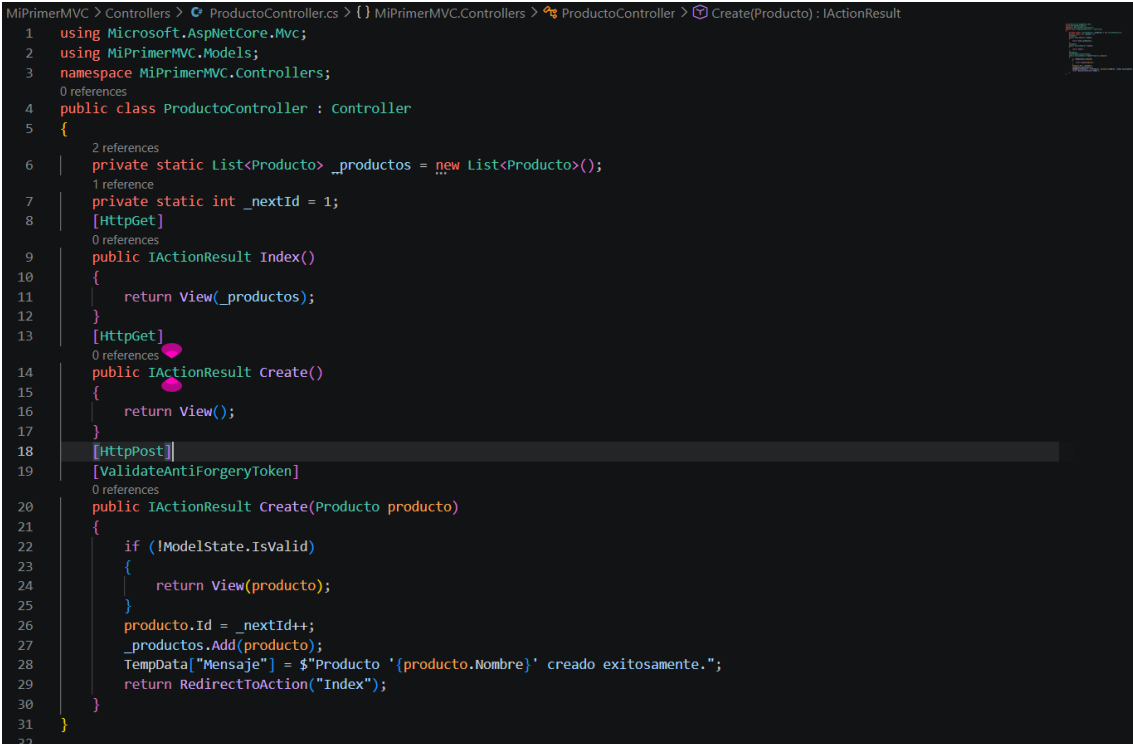
[ValidateAntiForgeryToken]

public IActionResult Create(Producto producto)

Esta acción se ejecuta cuando el usuario envía el formulario de creación. El proceso realizado es el siguiente:

1. Valida la información ingresada mediante ModelState.IsValid.
2. Asigna automáticamente un identificador (ID) al producto.
3. Guarda el producto en la lista _productos.
4. Redirige a la vista Index, donde se muestra la lista actualizada junto con un mensaje de confirmación.

Captura de la interfaz Código del ProductoController en Visual Studio



```
1 using Microsoft.AspNetCore.Mvc;
2 using MiPrimerMVC.Models;
3 namespace MiPrimerMVC.Controllers;
4 public class ProductoController : Controller
5 {
6     private static List<Producto> _productos = new List<Producto>();
7     private static int _nextId = 1;
8     [HttpGet]
9     public IActionResult Index()
10    {
11        return View(_productos);
12    }
13    [HttpGet]
14    public IActionResult Create()
15    {
16        return View();
17    }
18    [HttpPost]
19    [ValidateAntiForgeryToken]
20    public IActionResult Create(Producto producto)
21    {
22        if (!ModelState.IsValid)
23        {
24            return View(producto);
25        }
26        producto.Id = _nextId++;
27        _productos.Add(producto);
28        TempData["Mensaje"] = $"Producto '{producto.Nombre}' creado exitosamente.";
29        return RedirectToAction("Index");
30    }
31 }
32
```

Ilustración 3 Código del ProductoController en Visual

Studio

V. Vistas Razor

1. Vista Index (lista de productos)

La vista Index.cshtml recibe la List<Producto> y muestra una tabla con todos los productos registrados. También incluye un mensaje de alerta cuando la lista está vacía.

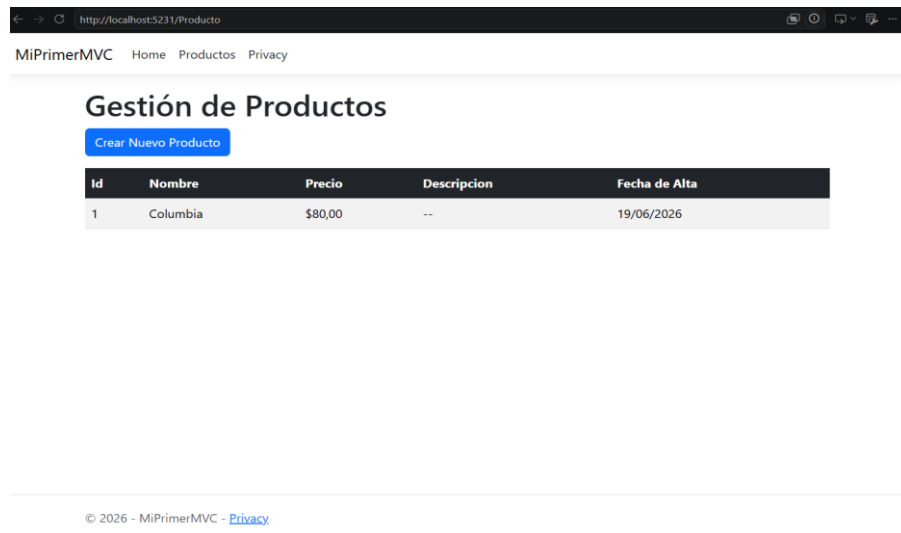


Ilustración 4 Muestra la tabla de los productos registrados

http://localhost:5231/Producto/Create

MiPrimerMVC Home Productos Privacy

Crear Producto

Nombre

El nombre es obligatorio

Precio

El precio es obligatorio

Descripción

Fecha de Alta

mm/dd/yyyy

La fecha es obligatoria

Crear Volver

Ilustración 5 Muestra los mensajes de alerta cuando los campos están vacíos.

2. Vista Create (formulario de creación)

La vista Create.cshtml contiene el formulario con AntiForgeryToken, validación client-side mediante asp-validation-for y resumen de errores con asp-validation-summary.

```

MiPrimerMVC > Views > Producto > Create.cshtml
1 @model Producto
2 @{ ViewData[Index: "Title"] = "Crear Producto"; }
3 <h1>Crear Producto</h1>
4 <hr />
5 <div class="row">
6     <div class="col-md-6">
7         <form asp-action="Create" method="post">
8             @Html.AntiForgeryToken()
9             <div asp-validation-summary="ModelOnly" class="text-danger"></div>
10            <div class="mb-3">
11                <label asp-for="Nombre" class="form-label"></label>
12                <input asp-for="Nombre" class="form-control" />
13                <span asp-validation-for="Nombre" class="text-danger"></span>
14            </div>
15            <div class="mb-3">
16                <label asp-for="Precio" class="form-label"></label>
17                <input asp-for="Precio" class="form-control" />
18                <span asp-validation-for="Precio" class="text-danger"></span>
19            </div>
20            <div class="mb-3">
21                <label asp-for="Descripcion" class="form-label"></label>
22                <textarea asp-for="Descripcion" class="form-control" rows="3"></textarea>
23                <span asp-validation-for="Descripcion" class="text-danger"></span>
24            </div>
25            <div class="mb-3">
26                <label asp-for="FechaAlta" class="form-label"></label>
27                <input asp-for="FechaAlta" class="form-control" type="date" />
28                <span asp-validation-for="FechaAlta" class="text-danger"></span>
29            </div>
30            <div class="mb-3">
31                <input type="submit" value="Crear" class="btn btn-success" />
32                <a asp-action="Index" class="btn btn-secondary">Volver</a></div>
33        </form>
34    </div>
35 </div>
36 @section Scripts {
37     @await Html.RenderPartialAsync(partialViewName: ". ValidationScriptsPartial");

```

Ilustración 6 Formulario vista create

VI. AntiForgeryToken

La protección contra ataques CSRF (Cross-Site Request Forgery) se implementó de dos formas:

1. En la vista (Create.cshtml)

[fp@Html.AntiForgeryToken\(\)](#)

Este helper genera un campo oculto con un token único enlazado a la cookie de sesión del usuario.

2. En el controlador

[HttpPost]

[ValidateAntiForgeryToken]

public IActionResult Create(Producto producto)

El atributo [ValidateAntiForgeryToken] verifica que el token enviado coincida con el esperado. Si no coincide, devuelve un error HTTP 400 (Bad Request).

VII. Validación Server-Side

La validación del lado del servidor se implementó en el controlador:

[HttpPost]

[ValidateAntiForgeryToken]

public IActionResult Create(Producto producto)

{

if (!ModelState.IsValid)

{

return View(producto); // Devuelve el formulario con errores

}

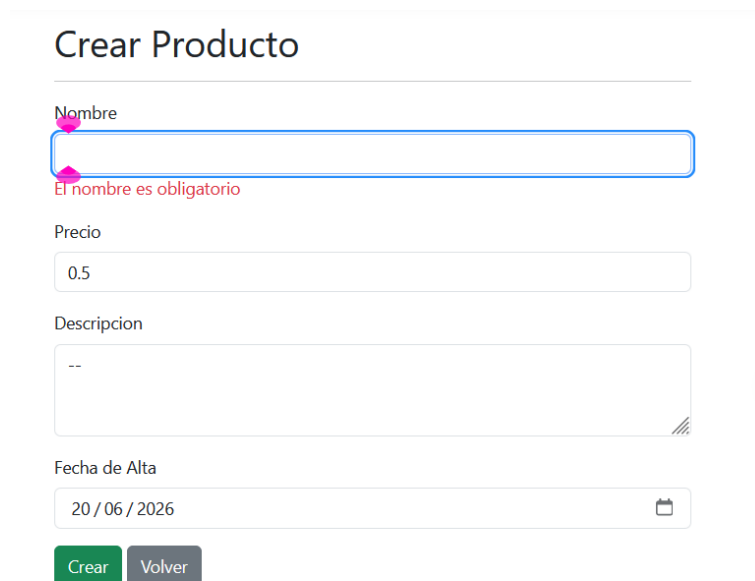
// ... guardar producto ...

Prueba de validación:

1. Se intentó enviar el formulario con todos los campos vacíos
2. El sistema detectó que ModelState.IsValid es false
3. Se devolvió la misma vista con los mensajes de error
4. El navegador mostró los errores en rojo: "El nombre es obligatorio", "El precio es obligatorio", etc.

Resultado: VALIDACIÓN CORRECTA

Envío de formulario con campo vacío



Crear Producto

Nombre

El nombre es obligatorio

Precio

0.5

Descripcion

--

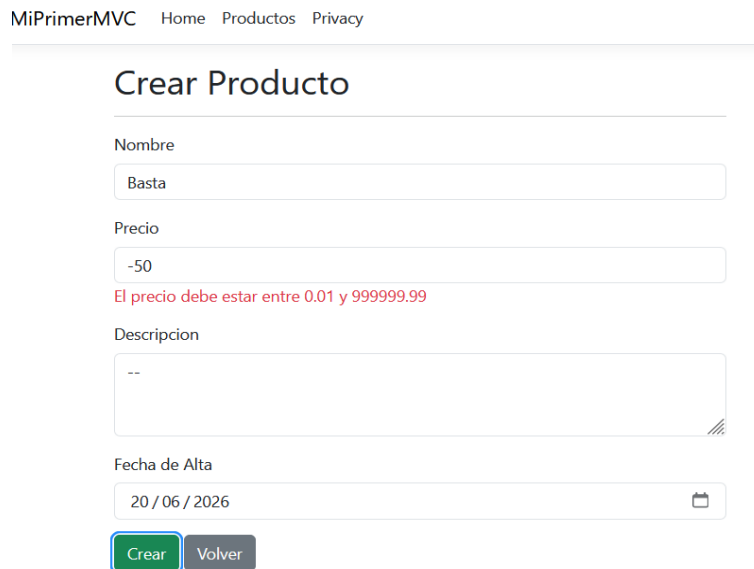
Fecha de Alta

20 / 06 / 2026

Crear Volver

Ilustración 7 Interfaz con un campo vacío

El sistema detectó que ModelState.IsValid es false



The screenshot shows a web application interface for creating a product. At the top, there is a navigation bar with links: 'MiPrimerMVC', 'Home', 'Productos', and 'Privacy'. Below the navigation bar, the main heading is 'Crear Producto'. The form contains several input fields: 'Nombre' with the value 'Basta', 'Precio' with the value '-50', and 'Descripcion' with the value '--'. A red error message is displayed below the 'Precio' field: 'El precio debe estar entre 0.01 y 999999.99'. At the bottom of the form, there is a 'Fecha de Alta' field with the value '20 / 06 / 2026' and a calendar icon. Below the form, there are two buttons: 'Crear' (highlighted with a green border) and 'Volver'.

Ilustración 8 Interfaz que detecto ModelState.IsValid

VIII. Conclusión

La práctica permitió aplicar los fundamentos del desarrollo web con ASP.NET Core MVC mediante la construcción de una aplicación para el registro y visualización de productos. Se logró implementar correctamente la estructura MVC, creando modelos con validaciones, controladores para procesar las solicitudes y vistas Razor para la interacción con el usuario.

Asimismo, se comprobó el funcionamiento de las validaciones mediante **Data Annotations** y **ModelState.IsValid**, asegurando que los datos ingresados cumplan las reglas definidas antes de ser almacenados. La incorporación de **AntiForgeryToken** permitió reforzar la seguridad de la aplicación frente a ataques CSRF, mientras que las validaciones del lado del cliente y del servidor mejoraron la experiencia del usuario y la confiabilidad del sistema.

En conclusión, el desarrollo de esta aplicación permitió adquirir experiencia práctica en la creación de aplicaciones web con ASP.NET Core MVC, comprendiendo la integración entre modelos, controladores y vistas, así como la importancia de la validación y la seguridad en el desarrollo de software web.